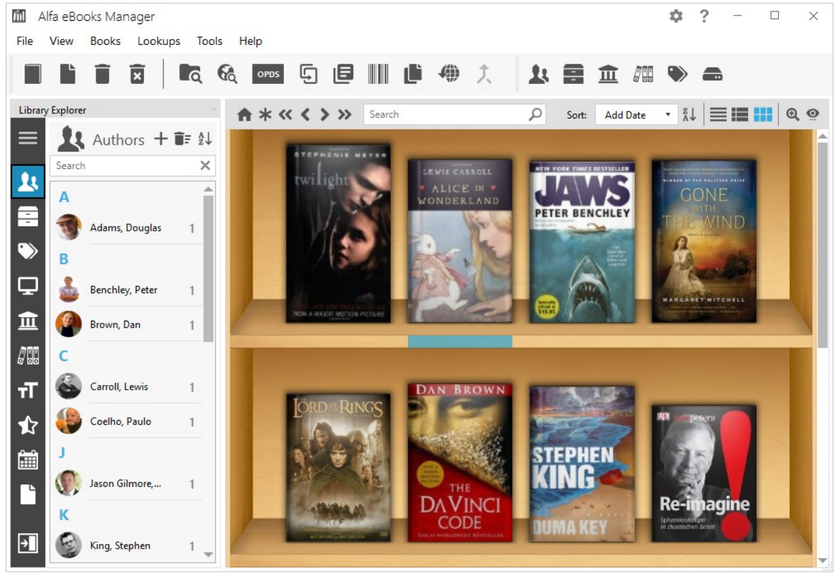
Héritage et Encapsulation

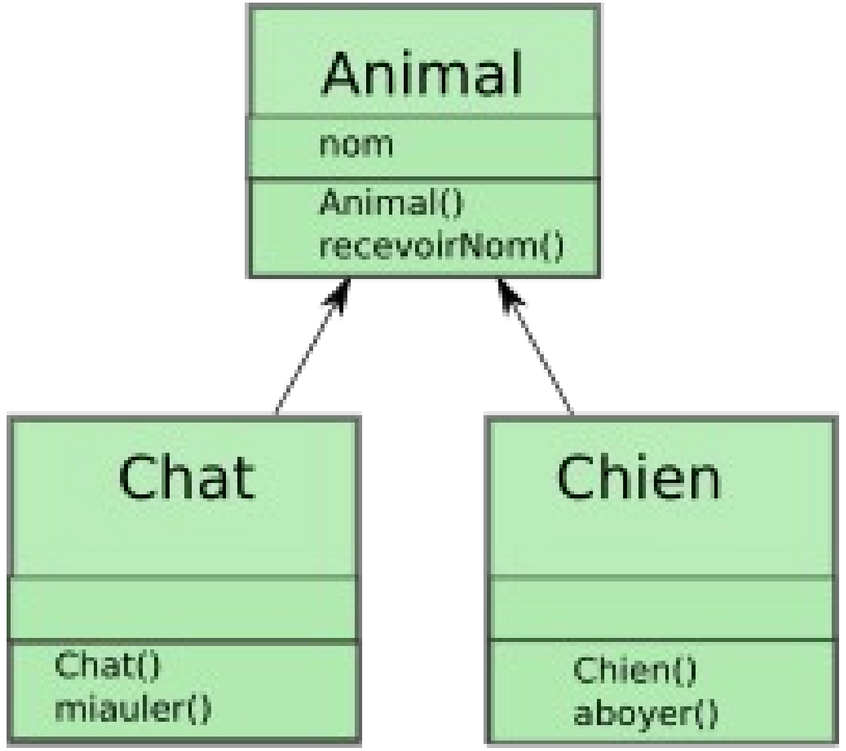
Dany Siriphol – CODA 1 ère année - 2025

Comment avoir un code réutilisable?



Héritage

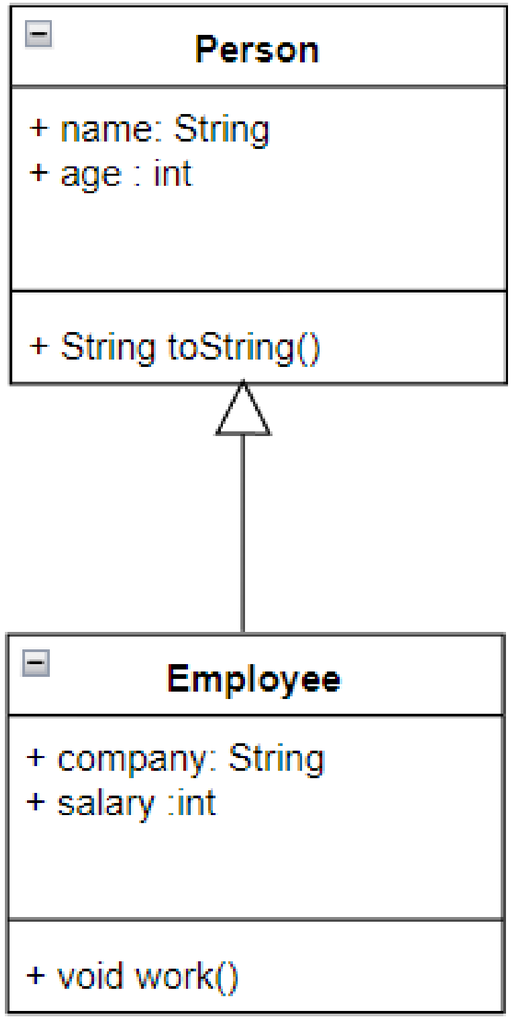
Si certaines classes partagent un ensemble de fonctionnalités,
il est possible de les réutiliser du code en servant de l’héritage



Héritage

L’héritage permet de définir une nouvelle classe à partir d’une classe existante.

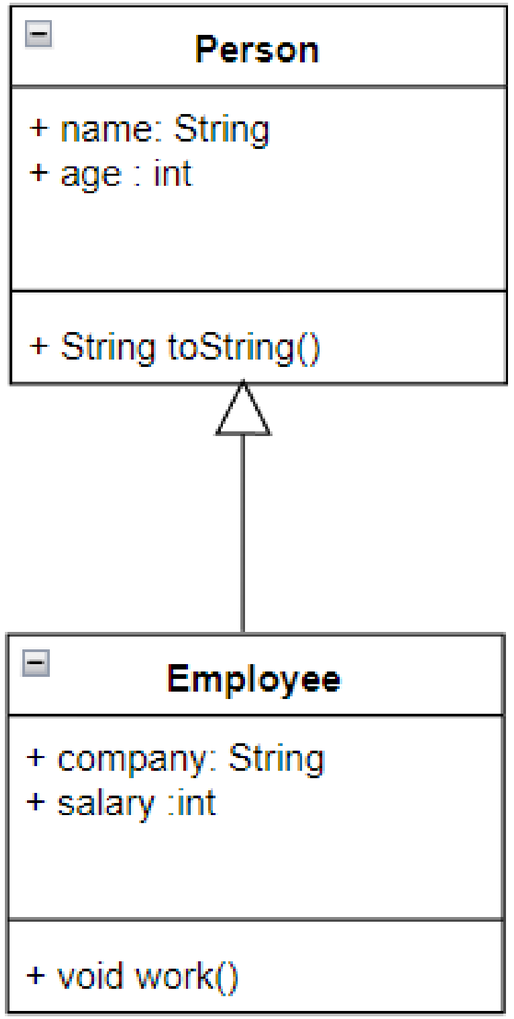
La classe qui hérite possédera toutes les **methodes** et **propriétés** de la classe parente



Héritage : Vocabulaire

La classe Employee hérite de la classe Person
La classe Employee étend la classe Person

La classe Person est la classe **parent** de Employee
La classe Employee est une classe **enfant** de Person



Héritage :

```java
public class Person {

    public String firstname;
    public String lastname;
    public int age;

    public Person(String firstname, String lastname) {
        this.firstname = firstname;
        this.lastname = lastname;
        this.age = 25;
    }

    public String toString()
    {
        return this.firstname;
    }
}
```

```java
public class Employee extends Person {

    public String company;
    public int salary;

    public Employee(String firstname, String lastname){
        super(firstname, lastname);
    }
}
```

Héritage : constructeur et super

La classe enfant peut définir son propre constructeur.

Le mot clé **super** permet d'accéder aux éléments de la classe parent (méthodes, propriétés)

Dans le constructeur d'une classe enfant, il est obligatoire d'appeler le constructeur du parent via super(...)

```
public class Employee extends Person {

    public String company;
    public int salary;

    public Employee(
            String firstname,
            String lastname){
        super(firstname, lastname);
    }
}
```

Héritage : redéfinir une méthode héritée

```java
public class Person {

    public String firstname;
    public String lastname;
    public int age;

    ...

    public void work() {
        this.age ++;
    }
}
```

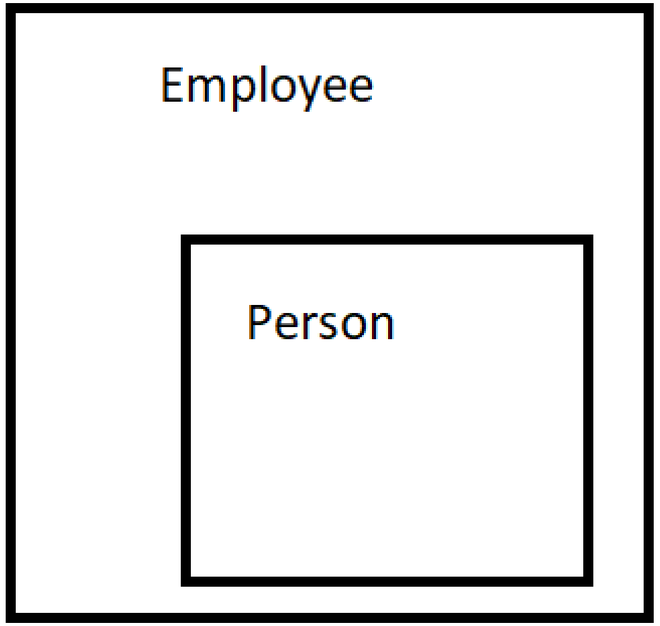
```java
public class Employee extends Person {

    public String company;
    public int salary;
    public int money;

    ...

    public void work() {
        super.work();
        this.money += salary;
    }
}
```

```java
var p = new Employee("John", "Doe");

p.work();
```

Héritage : Extensibilité

L’héritage permet de dériver une classe et lui ajouter des comportements via l’ajout de méthodes ou la redéfinition de méthodes existantes

Elle permet de réutiliser du code et d’éviter les redondances



Surcharge

La surcharge permet de créer plusieurs version d’une même méthode mais avec une signature différente

Les methodes de surcharge doivent porter le meme nom de méthode mais peuvent avoir un type de retour et des paramètres differents

```
public Person(String firstname, String lastname) {
    this.firstname = firstname;
    this.lastname = lastname;
    this.age = 25;
}

public Person(String firstname, String lastname, int age) {
    this.firstname = firstname;
    this.lastname = lastname;
    this.age = age;
}
```

Héritage et surcharge

Une classe enfant peut surcharger des méthodes du parent

La surcharge permet d’avoir différentes version d’une même méthode pour pouvoir s’adapter à différents contexte

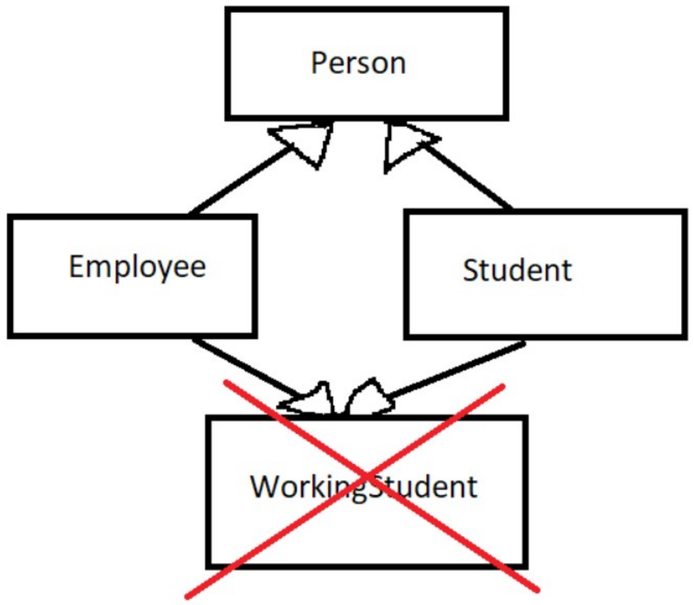
```java
public class Employee extends Person {

    public String company;
    public int salary;
    public int money;

    ...

    public void work() {
        super.work();
        this.money += salary;
    }

    public void work(Employee boss) {
        super.work();
        this.money += salary - boss.salary;
    }
```

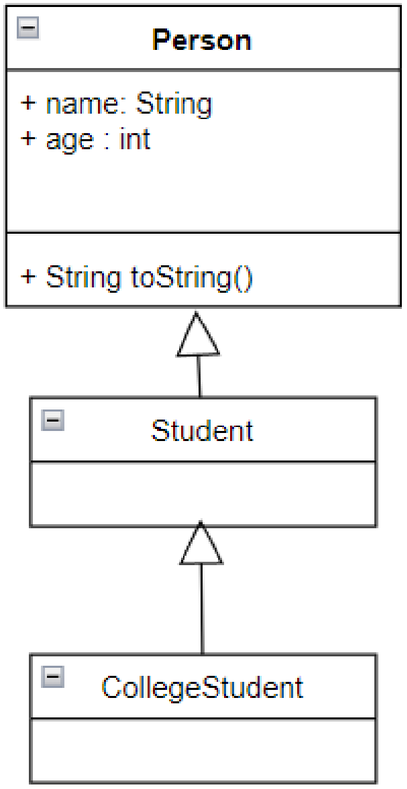
Héritage : Règles d’héritage

Une classe peut être hérite par plusieurs enfants

Une classe ne peut hériter que d’un parent
(pas d’héritage multiple en Java)

Il peut y’avoir plusieurs niveau d’heritage



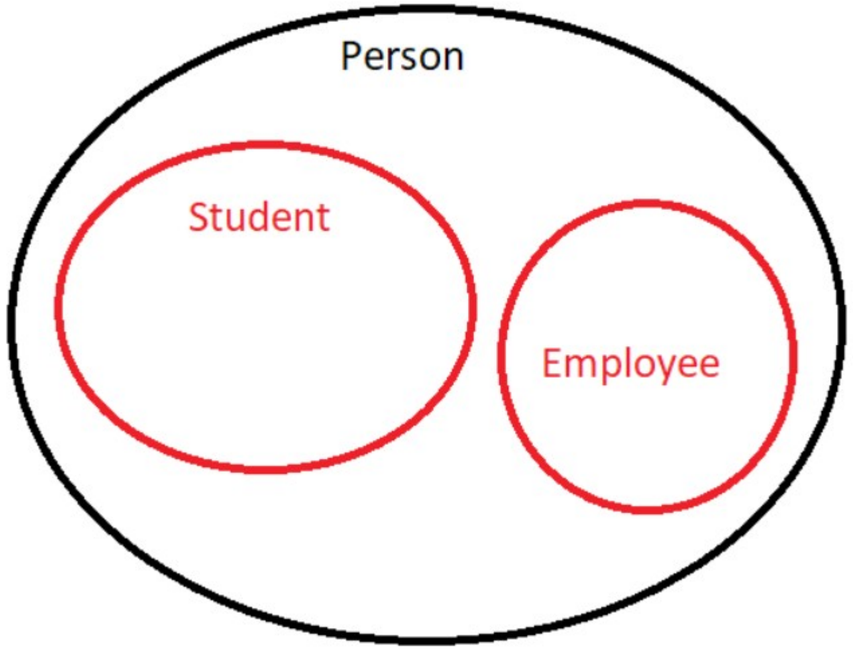


Héritage et typage

Compatibilité entre les classes

Une classe enfant possède le type de sa classe parent

Un Student est une Person
Un Employee est une Person



Compatibilité entre les classes

Les classes enfants **sont compatibles avec la classe parent**, on peut les substituer lors de l'utilisation d'instance

```
Person p1 = new Person("John", "Doe");
Person p2 = new Employee("John", "Doe");
```

Les classes parents ne sont pas compatible avec les classe enfant

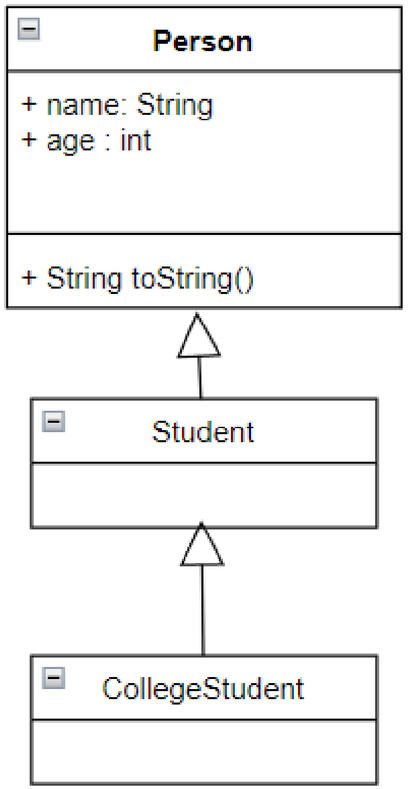
Une Person n'est pas forcement un Employee

```
Employee p1 = new Person("John", "Doe");
```

Compatibilité entre les classes

Avec plusieurs niveau d’heritage, une classe est compatible avec toutes ses surclasses

Un CollegeStudent est une Person



Visibilité et encapsulation

Dany Siriphol – CODA 1 ère année - 2025

Visibilité

Syntaxe

Lors de la déclaration d’une classe, il faut indiquer la visibilité des attributs et methodes. Cela définit qui peut accéder à ces éléments

Il existe 4 visibilités en java : public, protected, package, private

```java
public class Person {

    public String firstname;
    public String lastname;
    public int age;

    public Person(String firstname, String lastname) {
        this.firstname = firstname;
        this.lastname = lastname;
        this.age = 25;
    }
```

Visibilité

public

L’élément est visible par tout le code

protected

L’élément est visible par la classe actuelle et ses classes enfants

private

L’élément n’est visible que dans la classe actuelle

package (visiblité par défaut)

L’élément est visible par toutes les autres classes du package

Quelle visibilité choisir ?

public

Propriétés ou méthodes qui a pour objectif d’être utilisé et partagé à tout le code. Exemple : fonction de librairie, action de base sur un objet

protected

Propriétés ou méthodes qui ne doivent pas être utilisé à l’extérieur de la classe mais qui a la possibilité d’être modifié par une classe enfant. Cela permet d’avoir un morceau du code qui est flexible et peut etre redéfini ou surchargé

private

L’élément n’est utilisé que dans la classe et ne doit pas être utilisé par le code externe. Les propriétés et les méthodes qui relèvent du fonctionnement interne de la classe sont en général private

package (visiblité par défaut)

Faire collaborer des classes d’une même famille ensemble

Encapsulation

Définition

In software systems, encapsulation refers to the bundling of data with the mechanisms or methods that operate on the data. It may also refer to the limiting of direct access to some of that data, such as an object's components. Essentially, encapsulation prevents external code from being concerned with the internal workings of an object.

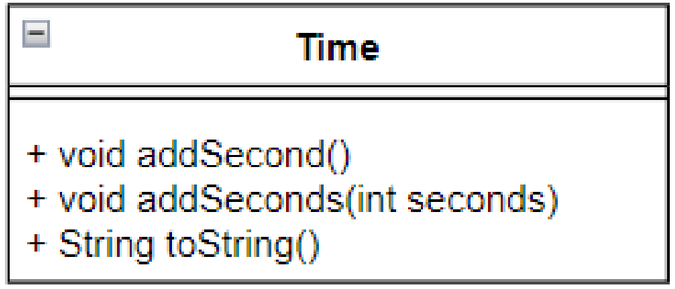
Encapsulation

L’encapsulation est l’un des piliers de la POO.

Cela consiste à définir dans une classe ce qui peut être utilisé à l’extérieur de ce qui ne doit pas l’être Le choix des visibilités est primordial.

L’encapsulation est le fait de définir comment doit être utilisé un objet, indépendamment de son fonctionnement interne.

```
public class Time
{
    private int seconds;
    private int minutes;
    private int hours;

    public Time(int hours, int minutes, int seconds)
    {
        this.seconds = seconds;
        this.minutes = minutes;
        this.hours = hours;
    }

    public void addSecond()
    {
        ...
    }
```



getter et setter

Définition

Les getter et setter sont des méthodes qui permettent de contrôler l’accès à une propriété privée

get : lire une propriété privée
set : modifier une propriété privée

```
private int seconds;

public int getSecond() { return this.seconds; }

public void setSecond(int seconds ) { this.seconds = seconds; }
```

getter et setter

Les getter permettent par exemple d’avoir une propriété accessible en lecture mais pas écriture

```
private int seconds;

public int getSecond() { return this.seconds; }
```

getter et setter

Les getter et setter permettent d’attacher un algorithme lors d’une lecture ou écriture. Par exemple pour effectuer des contrôles

```java
private int seconds;

public int getSecond() {
    return this.seconds % 60;
}

public void setSecond(int seconds ) {
    if ( seconds < 0 || seconds >= 60) {
        System.out.println("bruh");
        return;
    }
    this.seconds = seconds;
}
```

Exercices